SQE Mids Study Guide

Easy notes for all 5 lectures, with Urdu examples and small practice tools. Study one topic at a time.

CHAPTER 1 OF 5

Chapter 1: Introduction to Software Quality Engineering

Quality · Quality Types · Quality Control · Different views of Quality

1 What is Quality?

💡 **KEY POINT**

Quality = meeting the customer's requirements, at the **agreed cost**, within the **agreed time**.
Also: **“Fitness for purpose”** (Joseph Juran) and **customer satisfaction**.

- Three things together: **requirements + cost + time**.
- A good product that comes late or costs too much is **not** quality.

اردو مثال

آپ نے درزی سے کہا: "کل تک 2000 روپے میں کُرتا چاہیے۔" اگر کُرتا ٹھیک ناپ کا، 2000 میں اور کل تک مل گیا تو کوالٹی ہے۔ اگر خوبصورت ہے مگر ایک ہفتہ لیٹ آیا یا 3500 کا پڑا، تو کوالٹی نہیں۔

📍 **Where would you use this?** When a client says the product is “good” but it is late or over budget: check requirements, cost and time together.

2 Definitions of Quality

Who	Simple meaning
American Heritage Dictionary	A characteristic or attribute of something.
Juran	Fitness for purpose.
IEEE Glossary	Degree to which a system, component or process meets (1) **specified requirements** and (2) **customer needs or expectations**.
ISO 8402	The totality of features of a product/service that bear on its ability to satisfy **stated or implied needs**.
Gerald Weinberg	“Quality is **value to some person**.” So it is **subjective**: different people feel different quality in the same software.

اردو مثال (Weinberg)

ایک ہی موبائل ایپ طالب علم کو بہت اچھی لگتی ہے کیونکہ مفت ہے، لیکن دکاندار کو بری لگتی ہے کیونکہ اس میں بل بنانے کی سہولت نہیں۔ کوالٹی "کسی شخص کے لیے قدر" ہے۔

3 Software Quality (Roger Pressman)

💡 **KEY POINT**

Software quality = conformance to **(1) explicit functional & performance requirements**, **(2) documented development standards**, and **(3) implicit characteristics** expected from all professional software.

3 important points:

- Requirements are the **foundation**. No conformance to requirements = no quality.
- Standards guide how software is built. If you ignore them, lack of quality is almost sure.
- If software meets explicit requirements but fails **implicit** ones, quality is **suspect**.

اردو مثال

واضح (explicit): "ایپ میں لاگ اِن ہو۔" غیر واضح (implicit): کسی نے کہا نہیں مگر ہر کوئی چاہتا ہے کہ ایپ بار بار کریش نہ ہو اور پاس ورڈ محفوظ رہے۔ اگر ایپ لاگ اِن کر دے مگر روز کریش ہو تو کوالٹی مشکوک ہے۔

📍 **Where would you use this?** When writing a specification: also list the unwritten expectations such as stability and security.

4 Variation Control

KEY POINT

Variation is the **heart of quality control**. Goal: reduce the difference between **predicted** resources (staff, equipment, time) and **actual** resources used.

اردو مثال

آپ نے اندازہ لگایا کہ پروجیکٹ 3 ماہ اور 4 بندوں میں مکمل ہوگا۔ حقیقت میں 6 ماہ اور 7 بندے لگ گئے۔ یہ بڑا "فرق" (variation) ہے۔ اچھی ٹیم اس فرق کو کم سے کم رکھتی ہے۔

5 Quality Type 1: Quality of Design

KEY POINT

How well the software is **designed**. It is the quality the producer **intends** to give the customer (keeping customer requirements in mind).

- Design is only **5% of product cost** but has **70% influence** on quality, manufacturability, serviceability and market acceptance.
- Improving quality of design also improves quality of conformance.

اردو مثال

گھر کا نقشہ (design) بنانے پر کل خرچ کا بہت کم حصہ لگتا ہے، مگر اگر نقشہ ہی غلط ہو (ہوا اور روشنی کا انتظام نہ ہو) تو بعد میں مستری جتنا بھی اچھا کام کرے، گھر اچھا نہیں بنے گا۔

📍 **Where would you use this?** When a product misses its targets: decide whether the plan (design) or the build (conformance) is at fault.

6 Quality Type 2: Quality of Conformance

KEY POINT

How well the product made **matches the design/specification**. It focuses on **implementation**. If it matches fully, conformance is “excellent”.

- **Specification** = **targets** (ideal values) + **tolerances** (acceptable deviation).
- Exact target is hard to hit every time because of variation in **material, machine, men and process** (4 Ms).

اردو مثال

سموسے کا ڈیزائن: وزن 100 گرام (target)، 5 گرام کم یا زیادہ چلے گا (tolerance)۔ اگر 97 گرام نکلا تو ٹھیک، 85 گرام نکلا تو conformance خراب۔

7 Different views: Objective vs Perceived Quality

	Objective quality	Perceived quality
Meaning	Measured by instruments and expert ratings on product attributes	Overall **subjective** judgment compared to expectation
Example	PC speed, hard disk size, modem	Brand reputation, price, advertising, others' opinions
Includes aesthetics/brand?	No	Yes
Need to use product?	Yes (measure it)	No, you can form opinion without using it

> ⚠️ **NOTE**
> It is **perception**, not objective quality, that drives preference, satisfaction, loyalty, sales and profit. Example: it took **Google 3 years** after launch to be perceived as the superior search engine.

اردو مثال

دو موبائل کی ریم، بیٹری اور کیمرا بالکل برابر ہیں (objective برابر)۔ مگر لوگ مشہور برانڈ کو "زیادہ اچھا" سمجھتے ہیں کیونکہ اشتہار اور دوستوں کی رائے ہے (perceived مختلف)۔

📍 **Where would you use this?** When users distrust a technically good product: look at perceived quality.

8 Research: How objective quality affects perceived quality (Mitra & Golder)

- Studied **241 products**, **46 categories**, over **12 years**.
- A change in objective quality is fully reflected in perception only after about **6 years**.
- In the first year only about **20%** of the total effect shows.
- Effects are **bigger and faster for a decrease** in quality than for an equal increase (bad news travels fast).
- **High-reputation brands** get a "double advantage": rewarded **3 years quicker** for an increase, punished **1 year slower** for a decrease.

اردو مثال

اگر کسی مشہور دکان کی مٹھائی ایک دن خراب نکل آئے تو لوگ کچھ عرصہ معاف کر دیتے ہیں۔ نئی دکان کی مٹھائی بہتر بھی کر دیں تو لوگ سال ہا سال یقین نہیں کرتے۔

9 Quality Engineering

> 💡 **KEY POINT**
> The set of **operational, managerial and engineering activities** a company uses to make sure product quality characteristics are at the **nominal (required) levels**.

اردو مثال

فیکٹری میں مشین چلانا (operational)، منیجر کا منصوبہ بنانا (managerial) اور انجینئر کا مشین ٹھیک کرنا (engineering): تینوں مل کر یقینی بناتے ہیں کہ بسکٹ ہر بار ایک جیسا بنے۔

10 Quality Control (QC)

💡 **KEY POINT**

QC = processes and methods used to **monitor work** and see whether requirements are met. It focuses on **reviews and removal of defects before shipment**.

- **Objective:** minimize produced defects, increase product quality.
- Done by the same team that builds the product, **or** by a separate QC group/department.
- **Implementation approaches:** fully automated · entirely manual · combination of tools + humans.

اردو مثال

بیکری میں ڈبل روٹی پیک ہونے سے پہلے ایک بندہ ہر روٹی دیکھتا ہے۔ جلی ہوئی نکال دیتا ہے۔ یہ QC ہے: ڈلیوری سے پہلے نقص ڈھونڈنا۔

📍 **Where would you use this?** Before every release: tests and reviews that find and remove defects.

📝 Mid-exam style questions (Ch 1)

Define quality and software quality.

Difference between quality of design and quality of conformance?

Objective vs perceived quality?

What is QC? Its approaches?

CHAPTER 2 OF 5

Chapter 2: Software Quality Models

McCall · ISO 9126 · Boehm · FURPS · GQM

1 What is a Quality Model?

💡 **KEY POINT**

A quality model is a set of **characteristics and the relationships between them** that gives the basis for **specifying quality requirements** and **evaluating quality**.

Type	Model	Size
Standard models	McCall	11 factors, 23 criteria
	ISO/IEC 9126	6 characteristics, 27 sub-characteristics
	Boehm	7 factors, 15 criteria
Company/application specific	FURPS (Hewlett Packard)	5 factors (+ FURPS+)
	GQM approach	Goal → Question → Metric

اردو مثال

کسی لڑکے کو "اچھا" کہنے کے لیے ہر کوئی اپنا پیمانہ رکھتا ہے۔ Quality model ایک مشترکہ چیک لسٹ ہے تاکہ سب ایک ہی پیمانے سے سافٹ ویئر کو ناپیں۔

2 McCall's Factor–Criteria–Metrics Model

Level	Purpose	View
Factors	to **specify**	**External** view (users)
Criteria	to **build**	**Internal** view (developers)
Metrics	to **control**	Scale and method for measurement

- Quality **factor** = behavioural characteristic of a system (correctness, reliability, efficiency, testability, portability, maintainability…).
- Quality **criterion** = attribute of a factor related to development. Example: **Modularity** (cohesive components in one module) increases maintainability.
- McCall model is aimed at **system developers**, used **during development**. It does **not say much about functionality**.

اردو مثال

موٹر سائیکل: گاہک کہتا ہے "تیز اور کم پیٹرول پیئے" (factor، باہر کا نظارہ)۔ مکینک انجن کی ٹیوننگ اور وزن کم کرنا دیکھتا ہے (criteria، اندر کا نظارہ)۔ پھر کلومیٹر فی لیٹر ناپتا ہے (metric)۔

3 McCall's 11 Quality Factors

Product Revision

(ability to change)

- Maintainability — Can I **fix** it?
- Flexibility — Can I **change** it?
- Testability — Can I **test** it?

Product Transition

(adapt to new environments)

- Portability — use on **another machine**?
- Reusability — **reuse** parts?
- Interoperability — **interface** with another app?

Product Operation

(how it works)

- Correctness — does it do what I want?
- Reliability — accurately **all the time**?
- Efficiency — runs as well as it can?
- Integrity — is it **secure**?
- Usability — can I **run** it?

> 💡 **KEY POINT**
> Memory trick: Revision = **M-F-T** (3), Transition = **P-R-I** (3), Operation = **C-R-E-I-U** (5). Total 3+3+5 = 11.

اردو مثال: ایزی پیسہ جیسی ایپ

- Correctness: بیلنس بالکل صحیح دکھائے۔
- Reliability: ہر بار، رات 2 بجے بھی کام کرے۔
- Integrity: دوسرا بندہ میرے پیسے نہ دیکھ سکے۔
- Usability: دادی بھی استعمال کر سکے۔
- Portability: اینڈرائیڈ اور آئی فون دونوں پر چلے۔
- Interoperability: بینک سے جڑ جائے۔
- Maintainability: بگ ملے تو جلد ٹھیک ہو۔

📍 **Where would you use this?** Explaining a banking app's failures to a manager: wrong balance = correctness, always working = reliability.

4 McCall's Quality Criteria and Relationships

Criterion	Meaning
Accuracy	Precision of computations and outputs
Completeness	Full implementation of required functionality
Error Tolerance	Continuity of operations under adverse conditions
Expandability	Degree to which functions can be expanded
Hardware Independence	How little the software depends on hardware
Modularity	Highly independent modules
Simplicity	Ease of understanding the software

Relationships:

- Each factor is positively influenced by a set of criteria, and the **same criterion affects many factors**. Example: **Simplicity** impacts reliability, usability and testability.
- Improving one factor may **hurt** another. Example: portable code may be less efficient.
- Some factors **help** each other. Example: improving correctness increases reliability.

اردو مثال

کوڈ کو سادہ (simplicity) رکھنے سے بگ کم ہوتے ہیں (reliability)، نیا بندہ جلد سمجھتا ہے (usability for developers) اور ٹیسٹ لکھنا آسان ہوتا ہے (testability)۔ ایک چیز، تین فائدے۔

5 Trade-offs between Quality Factors

Pair	Relation	Why
Integrity vs Efficiency	**Inverse ↓**	Access control needs extra code and processing → longer run time, more storage.
Usability vs Efficiency	**Inverse ↓**	A better user interface needs a lot more code.
Maintainability vs Flexibility	**Direct ↑**	Well-structured code is easy to fix *and* easy to modify.
Correctness vs Reliability	**Direct ↑**	More correct → more reliable.
Portability vs Efficiency	**Inverse ↓**	Portable code may be less efficient.

اردو مثال

بینک کی ایپ میں OTP، فنگر پرنٹ اور پاس ورڈ سب لگا دیں تو سیکیورٹی (integrity) بہت اچھی، مگر ایپ کھلنے میں دیر لگتی ہے (efficiency کم)۔ یہ inverse تعلق ہے۔

📍 **Where would you use this?** When a client wants maximum security and maximum speed: explain the trade-off.

6 ISO / IEC 9126 Model (6 characteristics, 27 sub-characteristics)

- **Software quality characteristic** = set of attributes by which quality is described and evaluated.
- A characteristic is refined into **sub-characteristics**; each sub-characteristic is evaluated by **metrics**.

> 💡 **KEY POINT**
> Memory trick: **F-R-U-E-M-P** → Functionality, Reliability, Usability, Efficiency, Maintainability, Portability.

📍 **Where would you use this?** Reviewing a finished product against six characteristics.

7 Boehm's Quality Model (1978)

- Given by **Barry W. Boehm in 1978**. **7 factors, 15 criteria**.
- Three **primary uses** at the top level:

Primary use	Meaning	Sub-characteristics
As-is utility	How well the software can be used as it is	Ease of use, reliability, efficiency
Maintainability	Ease of identifying what needs to change	Testability, understandability, flexibility/modifiability
Portability	Ease of changing software for a new environment	—

> 💡 **KEY POINT**
> Boehm's 7 factors: Portability, Reliability, Efficiency, Human engineering (usability), Testability, Understandability, Modifiability.

اردو مثال

پرانا ٹی وی: "جیسا ہے ویسے چلانا" (as-is utility)، خراب ہو تو مستری کو آسانی سے پتہ چلے کہ کیا بدلنا ہے (maintainability)، اور دوسرے گھر یا ملک کی وولٹیج پر چلے (portability)۔

8 FURPS / FURPS+ (Hewlett Packard)

- Proposed by **Robert Grady (HP) in 1987**, later extended by **IBM as FURPS+**. FURPS is originally **company-specific**; FURPS+ is widely used in industry.

Letter	Meaning
Functionality	All system-wide functional requirements
Usability	Aesthetics, consistency, documentation
Reliability	Availability, accuracy, recoverability
Performance	Response time, resource consumption, start-up time, recovery time
Supportability	Can it be extended, adapted, corrected?
+ **Design** constraints	e.g. a relational database is required
+ **Implementation** constraints	Constrain coding: required standards, platform, language
+ **Interface** requirements	Must interact with an external item
+ **Physical** requirements	Hardware limits: shape, size, weight

اردو مثال: ہسپتال سافٹ ویئر

- F: مریض کا ریکارڈ بنانا۔ U: ڈاکٹر کے لیے آسان اسکرین۔ R: 24 گھنٹے چلے۔ P: ریکارڈ 2 سیکنڈ میں کھلے۔ S: بعد میں نئی لیب جوڑی جا سکے۔
- +Design: ڈیٹا بیس ریلیشنل ہو۔ +Implementation: Java میں لکھا جائے۔ +Interface: لیب مشین سے جڑے۔ +Physical: ٹیبلٹ کا وزن 500 گرام سے کم۔

📍 **Where would you use this?** Writing a requirements list: sort every item into F, U, R, P, S or a constraint.

9 GQM: Goal – Question – Metric

💡 **KEY POINT**

GQM is an approach to **software metrics** with 3 levels:

1. **Conceptual (Goal)**: list the major goals of the project/process.
2. **Operational (Question)**: from each goal, questions to find out if it is being met.
3. **Quantitative (Metric)**: what must be measured to answer the questions.

اردو مثال

Goal: امتحان میں اچھے نمبر۔ **Question:** میں روز کتنی دیر پڑھتا ہوں؟ کتنے ٹاپک مکمل ہوئے؟

Metric: روزانہ گھنٹے، مکمل ٹاپک کی تعداد، پریکٹس ٹیسٹ کا اسکور۔

📍 **Where would you use this?** When a manager says “improve quality”: turn it into a goal, questions and metrics.

10 Selecting and Prioritizing Quality Factors

- You **cannot maximize all factors** (see trade-offs). So choose the few that matter most for the project and the customer.
- Steps: list factors → decide the project's priorities → check conflicts → set metrics for the chosen ones.

📝 Mid-exam style questions (Ch 2)

List McCall's 11 factors in 3 groups.

Difference between factors, criteria, metrics?

Name the 6 characteristics of ISO 9126.

What is FURPS+? Who proposed it?

Explain GQM with an example.

CHAPTER 3 OF 5

Chapter 3: Software Quality Engineering Cost

Cost of Quality · Prevention · Appraisal · Failure · 1-10-100 rule · Quality Cost Conformance model

1 Cost of Quality

KEY POINT

Cost of quality = **all costs** spent to get quality or to do quality-related work. It is **not only** the price of making a good product; it is also the **cost of NOT making** a good product.

- Every time work is **redone**, cost of quality goes up (reworking an item, retesting a component).
- Any cost that would **not exist if quality were perfect** is part of the cost of quality.
- **Total quality cost** = Prevention + Appraisal + Failure (Internal + External).

اردو مثال: بریانی کی دکان

- Prevention: تازہ مصالحہ خریدنا اور باورچی کی ٹریننگ۔
- Appraisal: دیگ کھولنے سے پہلے چکھ کر دیکھنا۔
- Internal failure: چکھنے پر خراب نکلی تو دیگ پھینک دینا۔
- External failure: گاہک نے شکایت کی، پیسے واپس کیے۔

2 Prevention Cost

KEY POINT

Cost of activities that **stop defects from happening**. Cheapest way to manage quality: avoid defects in the first place.

- Examples: **quality planning, testing equipment, staff training, quality improvement process** (e.g. **Six Sigma**).

اردو مثال

موبائل کی دکان میں شروع سے ہی اچھا چارجر بیچنا اور عملے کو سمجھانا کہ کیسے استعمال کرنا ہے، تاکہ بعد میں شکایت ہی نہ آئے۔

3 Appraisal Cost

KEY POINT

Cost of **measuring, evaluating or auditing** products to check they meet standards. Also called **inspection cost**. Done to catch defects **before shipment**.

- Examples: **inspections, testing, maintenance of test equipment, supervision of testing/inspection**.

اردو مثال

امتحان سے پہلے ٹیچر کا ٹیسٹ لینا تاکہ کمزوری پتہ چل جائے۔ ٹیسٹ پر وقت اور پیسہ لگتا ہے، یہ appraisal ہے۔

4 Failure Cost (Internal & External)

	Internal failure	External failure
When?	Defect found **before** shipping to customer	**After** delivery/shipment
Examples	Scrap, rework, repair, re-testing, rejected products	Warranty, product return & replacement, help line support, legal liability
Note	Better appraisal → more internal failure cost found. It is the price to avoid external failure, which can be **devastating**.	Gives rise to **intangible** costs (image).

اردو مثال

درزی نے کُرتا گاہک کو دینے سے پہلے دیکھا کہ آستین چھوٹی ہے، دوبارہ سلائی کی (internal)۔ اگر گاہک نے گھر جا کر پہنا تو واپس لے آیا اور سب کو بتایا (external)، یہ زیادہ مہنگا پڑتا ہے۔

📍 **Where would you use this?** Reporting where the money went after a bad release (refunds, help line, rework).

5 Tangible vs Intangible Cost

Tangible	Intangible
Quantifiable, tied to an identifiable source. Seen **instantly** (materials, wages, equipment rent).	Not quantifiable. Not seen now, **effects come later**: lost productivity, lost customer goodwill, low morale, missed deadline, lost sales from bad reputation.
Prevention, appraisal, internal and external failure costs.	Can be **3 or 4 times greater** than tangible costs.

اردو مثال

خراب کھانا پھینکنے کا نقصان (tangible) اندازہ ہو جاتا ہے۔ مگر جو گاہک ناراض ہو کر دوبارہ نہیں آیا اور دس لوگوں کو بتا دیا، اس کا نقصان گنا نہیں جا سکتا (intangible)، اور بہت بڑا ہوتا ہے۔

6 Cost of Good Quality vs Cost of Poor Quality

- **Cost of GOOD quality** = Prevention + Appraisal (building systems and processes to reduce failure).
- **Cost of POOR quality** = Internal failure + External failure + intangible costs (because quality was poor).

7 The 1-10-100 Rule

> 💡 **KEY POINT**
> **$1** spent on **prevention** saves **$10** on **correction** and **$100** on **failure** costs.

اردو مثال

گھر بناتے وقت پائپ ٹھیک جوڑنے پر 1 روپیہ (روک تھام)۔ فرش بننے سے پہلے لیکیج پکڑی تو ٹھیک کرنے میں 10 روپے۔ فرش اور رنگ ہو جانے کے بعد پانی ٹپکا تو 100 روپے۔

📍 **Where would you use this?** Convincing management to pay for early reviews.

8 Quality Cost Conformance Model (ECL)

- An example of **constrained optimization**.
- Prevention + appraisal costs **rise** as conformance level rises.
- Failure costs **fall** as conformance level rises.
- So **total cost is U-shaped**.
- **Economic Conformance Level (ECL)** is reached where prevention + appraisal costs are equal to the internal + external failure costs (the lowest point of total).

اردو مثال

کھانے میں 100٪ پرفیکٹ بنانے کے لیے بہت مہنگا سامان اور انسپکشن لگے گا، اتنا کہ منافع ہی ختم۔ اور بالکل چیک نہ کریں تو شکایتیں اور نقصان۔ درمیان کا ایک "عقل مند" مقام ہے جہاں کل خرچ سب سے کم ہو۔ یہی ECL ہے۔

📍 **Where would you use this?** Deciding how much testing and inspection is enough.

9 Repair Cost of Defects

KEY POINT

The **later** a defect is found, the **more expensive** it is to fix. A mistake in requirements that reaches the customer costs far more than one caught in the requirement review.

اردو مثال

درزی کو ناپ لیتے وقت غلطی پتہ چل جائے تو صرف نشان بدلنا ہے۔ کپڑا کٹ جائے تو نیا کپڑا۔ سلائی کے بعد پتہ چلے تو پورا کُرتا ضائع۔ گاہک کے گھر پہنچ کر پتہ چلے تو بدنامی بھی۔

Mid-exam style questions (Ch 3)

Explain cost of quality with its categories.

Internal vs external failure cost?

Explain the 1-10-100 rule.

Explain the Quality Cost Conformance model.

Tangible vs intangible cost?

CHAPTER 4 OF 5

Chapter 4: Software Quality Assurance

Quality Assurance · Defect prevention · Defect reduction · (containment starts here, details in Ch 5)

1 Quality Assurance (QA)

💡 **KEY POINT**

QA = establishing a **framework of organizational procedures and standards** that lead to high quality software. "Assurance": if processes are followed, management can be **assured** of product quality.

History:

- **1916**: first formal QA and control function at **Bell Labs** (manufacturing).
- **1950s–60s**: programmers controlled their own product quality.
- **1970s**: QA standards first introduced in **military contract** software development.

QC (Ch 1)	QA (Ch 4)
Checks the **product**, removes defects before shipment	Builds the **process/framework** so quality is assured

اردو مثال

QC: بیکری میں تیار روٹی دیکھ کر جلی ہوئی نکالنا۔ QA: بیکری کے قواعد بنانا: صفائی کا نظام، آٹے کا معیار، تندور کا درجہ حرارت، تاکہ روٹی جلے ہی نہیں۔

2 Classification Scheme for QA (dealing with defects)

1. Defect Prevention
Stop faults from being
injected

2. Defect Reduction
Detect and
remove
faults (static + dynamic testing)

3. Defect Containment
Limit damage of failures (Chapter 5)

اردو مثال: مچھر

روک تھام: پانی جمع نہ ہونے دیں (مچھر پیدا ہی نہ ہوں)۔ کمی: جو مچھر موجود ہیں سپرے سے مار دیں۔

کنٹرول: مچھر دانی لگائیں تاکہ کاٹ بھی لیں تو نقصان کم ہو۔

3 Defect Prevention

> 💡 **KEY POINT**
> Prevent faults from being injected via **error blocking** or **error source removal**.

- **Error source removal**: find the root causes behind missing/incorrect human actions and fix them (e.g. correct human misconceptions).
- Focus is on **people and their conceptual mistakes**, which may lead to bad choice of methodology, language, algorithm or QA strategy → many fault injections.
- **3 techniques:** (1) Education and training, (2) Process conformance and standards enforcement, (3) Tools/technologies and techniques.

اردو مثال

اگر ہر نیا پروگرامر array کے index میں غلطی کرتا ہے تو بگ ٹھیک کرتے رہنے کے بجائے ٹریننگ دیں تاکہ غلط فہمی ہی ختم ہو جائے، یہ "error source removal" ہے۔

📍 **Where would you use this?** When the same coding mistakes repeat: remove the cause with training and standards.

4 Education and Training

Helps people control, manage and improve how they work and removes misconceptions. Focus on **4 areas**:

Area	Why
Product & domain knowledge	If people don't know the application domain, wrong solutions get built.
Tools (development/testing)	Tool knowledge helps build high-quality software.
General software development knowledge	E.g. weak requirements analysis → rework in design, coding, testing.
Development process knowledge	E.g. in incremental development, if people don't know how increments fit together, interface problems occur.

> 💡 **KEY POINT**
> Trick: **P-T-G-P** = Product, Tools, General, Process.

اردو مثال

جو پروگرامر بینکنگ نہیں جانتا وہ سود اور سروس چارج کے حساب میں غلطی کرے گا۔ پہلے اسے بینکنگ کی بنیادی باتیں سکھائیں۔

5 Process Standards and Tools / Technologies

- **Process conformance and standards enforcement**: make people follow the agreed process and standards, so mistakes are blocked.
- Appropriate **methodologies** reduce fault injection. Many problems of low-quality "**fat software**" can be solved by discipline and a return to essentials: "**lean software**".
- Specific **tools** help: e.g. a **syntax-directed editor** that automatically balances every "{" with "}" in C reduces syntax problems.

اردو مثال

ورڈ میں spell-check خود غلط الفاظ پر لال لکیر لگا دیتا ہے، اس لیے غلطی لکھی ہی نہیں جاتی۔ کوڈ ایڈیٹر بھی قوسین خودبند کر دیتا ہے۔

6 Defect Reduction: Static Testing → Inspection

> 💡 **KEY POINT**
> Defect reduction = **detect and remove faults** after they have been injected. Two categories: **Static testing** (no execution) and **Dynamic testing** (execute the program).

Software Inspection = critical examination of software artifacts (code, design, test cases, requirements, test plans) by **human inspectors** to find and fix faults.

- Critical reading and analysis of artifacts.
- Done by **multiple inspectors** through some coordination process.
- Faults are found **directly** by humans (individually or in group sessions).
- Faults found must be **removed** and the removal **verified**.
- Includes **planning and follow-up** activities.

اردو مثال

اخبار چھپنے سے پہلے تین پروف ریڈر مضمون الگ الگ پڑھتے ہیں اور پھر ملتے ہیں، یوں غلطیاں چھاپہ خانے سے پہلے پکڑی جاتی ہیں۔ پروگرام چلانے کی ضرورت نہیں پڑی۔

📍 **Where would you use this?** A formal review of requirements or code before testing starts.

7 Static Testing: Walkthrough & Desk Checking

	Inspection	Walkthrough	Desk check
Who	Several inspectors	Group	**One person**
How	Critical reading + coordination, planning, follow-up	Group code reading; procedures slightly different from inspection and uses a **different error-detection technique**	Reads the program, checks it against an **error list** and/or **walks test data** through it

> 💡 **KEY POINT**
> Desk check = a **one-person inspection or walkthrough**.

اردو مثال

امتحان کا پرچہ دینے سے پہلے آپ خود جوابات دوبارہ پڑھ کر چیک کریں (desk check)۔ دوست کے ساتھ بیٹھ کر ایک ایک سوال پر بات کریں (walkthrough)۔ استاد، ہیڈ اور ایک سینئر مل کر پرچے کا باقاعدہ جائزہ لیں (inspection)۔

8 Dynamic Testing: Black-box & White-box

> 💡 **KEY POINT**
> Dynamic testing = **executing the software** and **observing its behavior/outcome**. Testing is the most important and most common QA activity.

	Black-box (functional)	**White / clear-box (structural)**
Checks	Correct handling of **external functions**; behavior matches user expectations/specification	Correct implementation of **internal units, structures and relations**
Emphasis	Reduce functional problems met by **target customers**	Observe failures from internal implementation → detect and remove faults
Needs code?	No	Yes

اردو مثال

لفٹ: بٹن دبا کر دیکھنا کہ صحیح منزل پر رکتی ہے (black-box)۔ لفٹ کی مشین اور تاریں کھول کر اندر کی ہر چیز جانچنا (white-box)۔

📍 **Where would you use this?** Black-box: acceptance tests from the specification. White-box: tests that cover every branch.

📝 Mid-exam style questions (Ch 4)

Define QA. Give its brief history.

Explain the QA classification scheme.

Explain defect prevention techniques.

Inspection vs walkthrough vs desk checking?

Black-box vs white-box testing?

CHAPTER 5 OF 5

Chapter 5: Software Defect Containment

Fault tolerance · Recovery · Diversity techniques · Decision mechanisms · Fault Tree Analysis

1 Defect Containment: Why do we need it?

KEY POINT

Defect containment = **containing failures to local areas** and **limiting the damage**. (1) **Fault tolerance** breaks the cause-effect link between faults and failures so local faults don't cause global failures. (2) **Failure containment** avoids catastrophic results (death, injury, severe damage).

- Software defects **cannot be fully removed** because systems are large and complex.
- Remaining faults may be triggered in **rare conditions**. It is unrealistic to write test cases for all conditions or inspect all scenarios.
- So we **tolerate** those faults or **contain** the failures to reduce damage.

اردو مثال

بجلی جانا ہم نہیں روک سکتے، اس لیے UPS لگا لیتے ہیں تاکہ جب بجلی جائے تو کمپیوٹر بند ہو کر کام ضائع نہ ہو۔ بجلی کی خرابی (fault) موجود ہے مگر نقصان (failure) نہیں ہوا۔

2 Software Fault Tolerance

> 💡 **KEY POINT**
> **Fault tolerance / graceful degradation** = the system continues to operate properly when some components fail. If quality decreases, the decrease is **proportional to the severity** of the failure.

- Assumes the system has **unavoidable and undetectable faults** and makes provisions to operate correctly anyway.
- Faults are detected and recovery happens using various methods.
- All fault tolerance is based on **redundancy** (for detection and recovery).
- **Stages:** error detection → confinement → evaluation of propagation of erroneous info → recovery.

اردو مثال: گریس فل ڈیگریڈیشن

چار انجن والا جہاز اگر ایک انجن کھو دے تو گرتا نہیں، بس رفتار تھوڑی کم ہو جاتی ہے۔ جتنی خرابی، اتنی ہی کمی۔

3 Error Detection

- Starting point of fault tolerance: verification tests to detect errors.
- Uses **active redundancy**: redundant elements operating **simultaneously** to prevent, or allow recovery from, failures → more availability.
- **Forms of redundancy:** Hardware · Software · Time · Information.

Approach	How it detects
Replication	Uses **copies of the system**, gives the same input, compares outputs. If **results differ** → error detected.
Temporal	Watches an **expected time interval**. If the watch **expires with no response** → error detected (watchdog timer).

اردو مثال

Replication: ایک ہی حساب دو بندوں سے کروائیں؛ جواب مختلف آئے تو غلطی۔ Temporal: امتحان میں گھنٹی؛ وقت ختم ہوا اور جواب نہیں آیا تو مسئلہ۔

4 Recovery Types

Backward recovery	Forward recovery
Undoes executed actions and returns to an earlier **consistent state**, then continues from there.	When an abnormal state is found, **suspends** processing until better conditions; **ignores** part/all of the work and continues with what is assumed correct.
Needs a **recovery point** that saves information during execution.	Corrects the wrong behavior by sending **compensation information**.

اردو مثال

پیچھے کی طرف: ویڈیو گیم میں آپ مرے تو آخری سیو پوائنٹ سے دوبارہ شروع (backward)۔ آگے کی طرف: بینک نے غلطی سے 500 زیادہ کاٹ لیے تو پرانا لین دین ختم کرنے کے بجائے اگلے ہی دن 500 واپس ڈال دیے (forward، compensation)۔

📍 **Where would you use this?** Banking transfers: undo to a safe state, or correct with compensation.

5 Fault Injection (for fault tolerance assessment)

- Testing the software under **abnormal circumstances** with erroneous or invalid external inputs.
- Goal: inject faults and see if the system can **detect and recover** as specified.
- Results can lead to fixing individual bugs or discovering **design deficiencies** in fault tolerance.

اردو مثال

فائر ڈرل: جان بوجھ کر الارم بجا کر دیکھنا کہ لوگ صحیح نکلتے ہیں یا نہیں۔ اسی طرح سافٹ ویئر میں جان بوجھ کر غلط ان پٹ یا نیٹ ورک بند کر کے دیکھتے ہیں کہ نظام کیسے سنبھالتا ہے۔

6 Design Diversity vs Data Diversity

	Design diversity	Data diversity
Idea	Several **independently developed versions** to a common specification (different teams)	**One version** of software; **change the input data** slightly
Why works	Different versions unlikely to fail the same way	Software fails for certain input values; a small acceptable change may avoid it
Environment	Multiple version software	Multiple data representation
Cost	**High** (many variants)	**Cheaper**
Techniques	Recovery Block, N-Version, N-Self-Checking	Retry Block, N-Copy

اردو مثال

Design diversity: ایک ہی سوال تین مختلف استادوں سے حل کروانا۔ Data diversity: ایک ہی استاد کو سوال تھوڑا مختلف انداز میں پوچھنا (مثلاً 0.5 کی جگہ 1/2)۔

7 Recovery Blocks (RB)

- Introduced **1974 by Horning**; early implementations by **Randell (1975)** and **Hecht (1981)**.
- Uses **repeated execution (redundancy over time)**. Related to **sequential systems**.
- Parts: a conventional block + **acceptance test** + **primary alternate** + zero or more **stand-by spares**.
- If an alternate **crashes** (divide by zero, time limit exceeded) or **fails the acceptance test**, the **state is restored** to what it was before the primary started, then the next alternate runs.

```
ensure   acceptance test
by       primary alternate
else by  alternate 2
else by  alternate n
else     error
```

اردو مثال

پلان A: موٹر سائیکل پر جانا۔ اگر پنکچر ہو (acceptance test fail) تو پہلے حالت واپس (سامان وہیں) اور پلان B: رکشہ۔ وہ بھی نہ ہو تو پلان C: بس۔ آخر میں سب fail تو "error"۔

📍 **Where would you use this?** Critical code with a backup routine that runs when the first one fails its check.

8 N-Version Programming (NVP)

- Suggested by **Elmendorf (1972)**, developed by **Avizienis & Chen (1977–78)**.
- Uses **parallel redundancy**: N different versions fulfilling the same function run **in parallel**; the **voter (decision algorithm)** compares all outputs and selects the correct one (usually **majority vote**).
- Aim: **mask the effect of software flaws** in the module's output. Local failure in a few versions will not spoil the global result.
- Compared with RB, NVP is a **static** technique. **Voter is the fundamental difference** between NVP and RB.
- Needs **lots of development effort** and teams (high cost).

```
run Version 1, Version 2, ..., Version n
if (Decision Mechanism (Result1, Result2, ..., Result n))
     return Result
else failure exception
```

اردو مثال

تین ڈاکٹروں سے الگ الگ رائے لیں۔ دو کہیں "ٹائیفائیڈ" اور ایک "ملیریا" تو اکثریت کی بات مانتے ہیں۔ ایک ڈاکٹر کی غلطی سے مریض کو نقصان نہیں ہوا۔

📍 **Where would you use this?** Systems where several independent versions compute the answer and a voter decides.

9 N-Self-Checking Programming (NSCP)

- Uses multiple versions combined with **structural variations of RB and NVP**.
- Versions **and acceptance tests** are developed **independently** from common requirements.
- **Main difference from RB:** a **separate acceptance test for each version**.
- Versions and tests can run sequentially or in parallel; the output is taken from the **highest-ranking version that passes its own acceptance test**.

اردو مثال

تین طلبہ اپنا اپنا جواب لکھتے ہیں اور ہر کوئی اپنا جواب خود چیک بھی کرتا ہے۔ جو سب سے سینئر طالب علم کا جواب اس کی اپنی چیکنگ میں پاس ہو، وہی جمع ہوگا۔

10 Retry Blocks (RtB)

- By **Ammann & Knight (1987–88)**. A modification of the recovery block that uses **data diversity** instead of design diversity.
- **Same algorithm**, **different data**: if the first try fails the acceptance test, **re-express the input** (equivalent or nearly equivalent form) and run again.
- Repeats until the test passes or the **deadline expires**; then a **backup algorithm** runs with the **original input**; if that fails → failure exception.

```
ensure   Acceptance Test
by       Primary Algorithm (Original Input)
else by  Primary Algorithm (Re-expressed Input)
else by  Primary Algorithm (Re-expressed Input)
... [Deadline Expires]
else by  Backup Algorithm (Original Input)
else     failure exception
```

اردو مثال

کیلکولیٹر 0 پر فیل ہوتا ہے تو آپ 0 کی جگہ 0.0001 ڈال کر دوبارہ حساب لگاتے ہیں۔ جواب تقریباً وہی آتا ہے۔ طریقہ وہی، ڈیٹا ذرا بدلا ہوا۔

11 N-Copy Programming (NCP)

- The **data-diverse version of NVP**.
- Uses one or more **Data Re-expression Algorithms (DRA)** and **at least two copies** of the same program.
- Inputs go through the DRAs → copies run **in parallel** on the re-expressed data → a **Decision Mechanism (DM)** picks the best result, if there is one.
- Parts: **executive**, 1 to n DRAs, n copies, and a DM. The executive orchestrates the steps.

```
run DRA 1, DRA 2, ..., DRA n
run Copy 1(result of DRA 1), Copy 2(result of DRA 2), ..., Copy n(...)
if (Decision Mechanism (Result 1, Result 2, ..., Result n))
     return Result
(else error occurs)
```

اردو مثال

ایک ہی کاریگر کی تین نقلیں (copies)، ہر ایک کو ناپ تھوڑے مختلف انداز میں دیا (cm، انچ، گز)۔
آخر میں منیجر (DM) دیکھتا ہے کہ کس کا نتیجہ سب سے صحیح ہے۔

	Design diversity	**Data diversity**
Sequential (try one by one)	Recovery Block	Retry Block
Parallel (vote/decide)	N-Version Programming	N-Copy Programming

12 Decision Mechanism

Voter
Compares results from **two or more** variants and decides the correct result, if any exists.

Comparator
A voter when there are exactly **two results** .

Acceptance test
Checks the **final result** . Doesn't matter which alternative produced it; only that it is **within acceptable limits** . Must be **simple** .

> ⚠ **NOTE**
> A badly designed acceptance test causes **two flaws**: (1) **rejecting a correct** alternative, (2) **accepting an incorrect** alternative.

اردو مثال

ٹیچر کا چیک: "جواب 0 سے 100 کے درمیان ہو" (آسان acceptance test)۔ اگر کوئی غلط جواب 40 لکھ دے تو بھی پاس ہو جائے گا (incorrect accepted)۔ اگر ٹیسٹ بہت سخت ہو تو صحیح جواب 101 بھی رد ہو سکتا ہے (correct rejected)۔

13 Failure Containment & Fault Tree Analysis (FTA)

> 💡 **KEY POINT**
> Even with prevention and tolerance, faults remain. Can we (a) **prevent accidents** and (b) **reduce their damage**? We can't prevent all accidents, but we can **analyze hazards** and **limit damage**.

- FTA was developed in **1962 at Bell Labs by H.A. Watson**, under a **US Air Force** contract, to evaluate the **Minuteman I ICBM** launch control system.
- FTA is a **top-down, deductive** failure analysis: start from an undesired state and use **Boolean logic (AND, OR)** to combine lower-level events.
- Used in **safety engineering and reliability engineering** to find the probability of a safety accident or a system-level failure.
- **Steps:** identify possible failures → list events that cause the accident → draw an **upside-down tree** (nodes = failures; edges = AND/OR).
- **Uses of the tree:** accident **elimination**, **reduction**, **control**.

Gate	Top event happens when...	Probability
OR	**any one** child happens	$1 - (1-p1)(1-p2)\ldots$
AND	**all** children happen	$p1 \times p2 \times \ldots$

اردو مثال: گاڑی سٹارٹ نہیں ہو رہی

ٹاپ: "گاڑی سٹارٹ نہیں" ← OR (بیٹری ڈیڈ، یا پیٹرول ختم، یا سیلف خراب)۔ ہر وجہ کے نیچے اور وجوہات۔ ہمیں درخت سے پتہ چلتا ہے کہ سب سے آسان راستہ کون سا ہے جس سے حادثہ ہو سکتا ہے، پھر وہیں حفاظت لگاتے ہیں۔

📍 **Where would you use this?** Analysing how a payment or control system could fail and where to add protection.

📝 Mid-exam style questions (Ch 5)

What is fault tolerance / graceful degradation?

Explain backward and forward recovery.

Explain Recovery Block with syntax.

NVP vs Recovery Block?

Design vs data diversity? Name techniques.

What is FTA? Where and when was it developed?

One-Page Cheat Sheet

Ch 1: Quality basics

- Quality = requirements + agreed cost + agreed time; fitness for purpose (Juran)
- IEEE: meets specified requirements + user needs. ISO 8402: stated or implied needs. Weinberg: value to some person
- Pressman: explicit req + standards + implicit characteristics
- Variation control: predicted vs actual resources
- Design (5% cost, 70% influence) vs Conformance (matches design; target ± tolerance)
- Objective (measured) vs Perceived (subjective; drives sales; Google 3 yrs)
- Mitra & Golder: 241 products, 46 categories, 12 yrs; 20% in year 1; full ~6 yrs; drops faster; high brand: +3 yrs quicker, −1 yr slower
- QC: monitor work, remove defects before shipment; automated/manual/both

Ch 2: Models

- McCall 11/23: factors (specify, external) · criteria (build, internal) · metrics (control)
- Revision: Maintainability, Flexibility, Testability
- Transition: Portability, Reusability, Interoperability
- Operation: Correctness, Reliability, Efficiency, Integrity, Usability
- Trade-offs: integrity↔efficiency (inverse), usability↔efficiency (inverse), maintainability↔flexibility (direct)
- ISO 9126 (6/27): Functionality, Reliability, Usability, Efficiency, Maintainability, Portability
- Boehm 1978 (7/15): as-is utility, maintainability, portability
- FURPS (Grady, HP, 1987; IBM FURPS+): Functionality, Usability, Reliability, Performance, Supportability + design, implementation, interface, physical
- GQM: Goal (conceptual) → Question (operational) → Metric (quantitative)

Ch 3: Cost

- Total = Prevention + Appraisal + Failure (internal + external)
- Prevention: planning, training, Six Sigma. Appraisal: inspection, testing, test equipment
- Internal: scrap, rework, repair, retest, rejects. External: warranty, returns, help line, legal
- Intangible (image, goodwill) can be 3–4× tangible
- Good quality cost = prevention + appraisal; poor = failures + intangible
- 1-10-100: $1 prevent, $10 correct, $100 failure
- ECL: total cost is U-shaped; optimum at the lowest point

Ch 4: QA

- QA = framework of procedures and standards. 1916 Bell Labs; 50s–60s programmers; 70s military
- Prevention → Reduction → Containment
- Prevention: error blocking / error source removal; training (product, tools, general, process), standards, tools (lean software, syntax-directed editor)
- Reduction: Static (inspection, walkthrough, desk check) · Dynamic (black-box, white-box)
- Inspection: multiple human inspectors + planning + follow-up. Desk check: one-person
- Black-box: external functions vs spec. White-box: internal structure

Ch 5: Containment

- Defects can't be fully removed → tolerate faults, contain failures
- Fault tolerance = graceful degradation, based on redundancy (hardware, software, time, information)
- Stages: detection → confinement → propagation evaluation → recovery
- Detection: replication (compare outputs), temporal (timer)
- Recovery: backward (recovery point, undo) · forward (continue, compensation)
- Fault injection: test under abnormal input
- Design diversity (costly): Recovery Block (1974 Horning; AT + state restore) · NVP (Elmendorf 1972; voter; static) · NSCP (own AT per version)
- Data diversity (cheaper, one version): Retry Block (Ammann & Knight; re-express input, deadline, backup) · NCP (DRA + copies + DM)
- AT flaws: reject correct / accept incorrect
- FTA: 1962 Bell Labs, H.A. Watson, Minuteman I; top-down, Boolean AND/OR; eliminate/reduce/control accidents